Tables (18)

Name	Type	Schema
assignments		CREATE TABLE assignments (id INTEGER PRIMARY KEY AUTOINCREMENT, class_id INTEGER NOT NULL, title VARCHAR(200) NOT NULL, description TEXT, deadline DATETIME NOT NULL, max_file_size INTEGER DEFAULT 10, allowed_file_types VARCHAR(200), created_at DATETIME DEFAULT CURRENT_TIMESTAMP, is_overdue, FOREIGN KEY (class_id) REFERENCES classes(id) ON DELETE CASCADE)
id	INTEGER	"id" INTEGER
class_id	INTEGER	"class_id" INTEGER NOT NULL
title	VARCHAR(200)	"title" VARCHAR(200) NOT NULL
description	TEXT	"description" TEXT
deadline	DATETIME	"deadline" DATETIME NOT NULL
max_file_size	INTEGER	"max_file_size" INTEGER DEFAULT 10
allowed_file_types	VARCHAR(200)	"allowed_file_types" VARCHAR(200)
created_at	DATETIME	"created_at" DATETIME DEFAULT CURRENT_TIMESTAMP
is_overdue		"is_overdue"
class_students		CREATE TABLE class_students (class_id INTEGER NOT NULL, student_id INTEGER NOT NULL, grade REAL, PRIMARY KEY (class_id, student_id), FOREIGN KEY (class_id) REFERENCES classes(id) ON DELETE CASCADE, FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE)
class_id	INTEGER	"class_id" INTEGER NOT NULL
student_id	INTEGER	"student_id" INTEGER NOT NULL
grade	REAL	"grade" REAL
classes		CREATE TABLE classes (id INTEGER PRIMARY KEY AUTOINCREMENT, class_code VARCHAR(50) NOT NULL UNIQUE, class_name VARCHAR(100) NOT NULL, course_id INTEGER NOT NULL, lecturer_id INTEGER NOT NULL, semester_id INTEGER NOT NULL, status VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active', 'completed')), FOREIGN KEY (course_id) REFERENCES courses(id), FOREIGN KEY (lecturer_id) REFERENCES lecturers(id), FOREIGN KEY (semester_id) REFERENCES semesters(id))
id	INTEGER	"id" INTEGER
class_code	VARCHAR(50)	"class_code" VARCHAR(50) NOT NULL UNIQUE
class_name	VARCHAR(100)	"class_name" VARCHAR(100) NOT NULL
course_id	INTEGER	"course_id" INTEGER NOT NULL
lecturer_id	INTEGER	"lecturer_id" INTEGER NOT NULL
semester_id	INTEGER	"semester_id" INTEGER NOT NULL
status	VARCHAR(20)	"status" VARCHAR(20) DEFAULT 'active' CHECK("status" IN ('active', 'completed'))
course_types		CREATE TABLE course_types (id INTEGER PRIMARY KEY AUTOINCREMENT, type_name VARCHAR(50) NOT NULL UNIQUE)
id	INTEGER	"id" INTEGER
type_name	VARCHAR(50)	"type_name" VARCHAR(50) NOT NULL UNIQUE
courses		CREATE TABLE courses (id INTEGER PRIMARY KEY AUTOINCREMENT, course_type_id INTEGER NOT NULL, course_code VARCHAR(20) NOT NULL UNIQUE, course_name VARCHAR(100) NOT NULL, credits INTEGER DEFAULT 0, description TEXT, FOREIGN KEY (course_type_id) REFERENCES course_types(id))
id	INTEGER	"id" INTEGER
course_type_id	INTEGER	"course_type_id" INTEGER NOT NULL
course_code	VARCHAR(20)	"course_code" VARCHAR(20) NOT NULL UNIQUE
course_name	VARCHAR(100)	"course_name" VARCHAR(100) NOT NULL
credits	INTEGER	"credits" INTEGER DEFAULT 0

Name	Type	Schema
description	TEXT	"description" TEXT
feedbacks		CREATE TABLE feedbacks (id INTEGER PRIMARY KEY AUTOINCREMENT, submission_id INTEGER NOT NULL, lecturer_id INTEGER NOT NULL, content TEXT NOT NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (submission_id) REFERENCES submissions(id) ON DELETE CASCADE, FOREIGN KEY (lecturer_id) REFERENCES lecturers(id))
id	INTEGER	"id" INTEGER
submission_id	INTEGER	"submission_id" INTEGER NOT NULL
lecturer_id	INTEGER	"lecturer_id" INTEGER NOT NULL
content	TEXT	"content" TEXT NOT NULL
created_at	DATETIME	"created_at" DATETIME DEFAULT CURRENT_TIMESTAMP
lecturer_quotas		CREATE TABLE lecturer_quotas (id INTEGER PRIMARY KEY AUTOINCREMENT, lecturer_id INTEGER NOT NULL, semester_id INTEGER NOT NULL, course_type_id INTEGER NOT NULL, max_students INTEGER NOT NULL, current_students INTEGER DEFAULT 0, FOREIGN KEY (lecturer_id) REFERENCES lecturers(id), FOREIGN KEY (semester_id) REFERENCES semesters(id), FOREIGN KEY (course_type_id) REFERENCES course_types(id), UNIQUE(lecturer_id, semester_id, course_type_id))
id	INTEGER	"id" INTEGER
lecturer_id	INTEGER	"lecturer_id" INTEGER NOT NULL
semester_id	INTEGER	"semester_id" INTEGER NOT NULL
course_type_id	INTEGER	"course_type_id" INTEGER NOT NULL
max_students	INTEGER	"max_students" INTEGER NOT NULL
current_students	INTEGER	"current_students" INTEGER DEFAULT 0
lecturers		CREATE TABLE lecturers (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL UNIQUE, department VARCHAR(100), faculty VARCHAR(100), degree VARCHAR(50), bio TEXT, max_thesis INTEGER DEFAULT 0, max_project INTEGER DEFAULT 0, current_thesis INTEGER DEFAULT 0, current_project INTEGER DEFAULT 0, is_active INTEGER DEFAULT 1, FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE)
id	INTEGER	"id" INTEGER
user_id	INTEGER	"user_id" INTEGER NOT NULL UNIQUE
department	VARCHAR(100)	"department" VARCHAR(100)
faculty	VARCHAR(100)	"faculty" VARCHAR(100)
degree	VARCHAR(50)	"degree" VARCHAR(50)
bio	TEXT	"bio" TEXT
max_thesis	INTEGER	"max_thesis" INTEGER DEFAULT 0
max_project	INTEGER	"max_project" INTEGER DEFAULT 0
current_thesis	INTEGER	"current_thesis" INTEGER DEFAULT 0
current_project	INTEGER	"current_project" INTEGER DEFAULT 0
is_active	INTEGER	"is_active" INTEGER DEFAULT 1
notifications		CREATE TABLE notifications (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL, title VARCHAR(200) NOT NULL, content TEXT, related_id INTEGER, is_read INTEGER DEFAULT 0, created_at DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE)
id	INTEGER	"id" INTEGER
user_id	INTEGER	"user_id" INTEGER NOT NULL
title	VARCHAR(200)	"title" VARCHAR(200) NOT NULL
content	TEXT	"content" TEXT
related_id	INTEGER	"related_id" INTEGER

Name	Type	Schema
is_read	INTEGER	"is_read" INTEGER DEFAULT 0
created_at	DATETIME	"created_at" DATETIME DEFAULT CURRENT_TIMESTAMP
published_reports		CREATE TABLE published_reports (id INTEGER PRIMARY KEY AUTOINCREMENT, student_id INTEGER NOT NULL, class_id INTEGER NOT NULL, topic_id INTEGER NOT NULL, file_url VARCHAR(500), source_code_url VARCHAR(500), published_at DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (student_id) REFERENCES students(id), FOREIGN KEY (class_id) REFERENCES classes(id), FOREIGN KEY (topic_id) REFERENCES topics(id))
id	INTEGER	"id" INTEGER
student_id	INTEGER	"student_id" INTEGER NOT NULL
class_id	INTEGER	"class_id" INTEGER NOT NULL
topic_id	INTEGER	"topic_id" INTEGER NOT NULL
file_url	VARCHAR(500)	"file_url" VARCHAR(500)
source_code_url	VARCHAR(500)	"source_code_url" VARCHAR(500)
published_at	DATETIME	"published_at" DATETIME DEFAULT CURRENT_TIMESTAMP
registrations		CREATE TABLE registrations (id INTEGER PRIMARY KEY AUTOINCREMENT, student_id INTEGER NOT NULL, lecturer_id INTEGER NOT NULL, course_type_id INTEGER NOT NULL, semester_id INTEGER NOT NULL, status VARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending', 'approved', 'rejected')), registered_at DATETIME DEFAULT CURRENT_TIMESTAMP, processed_at DATETIME, FOREIGN KEY (student_id) REFERENCES students(id), FOREIGN KEY (lecturer_id) REFERENCES lecturers(id), FOREIGN KEY (course_type_id) REFERENCES course_types(id), FOREIGN KEY (semester_id) REFERENCES semesters(id))
id	INTEGER	"id" INTEGER
student_id	INTEGER	"student_id" INTEGER NOT NULL
lecturer_id	INTEGER	"lecturer_id" INTEGER NOT NULL
course_type_id	INTEGER	"course_type_id" INTEGER NOT NULL
semester_id	INTEGER	"semester_id" INTEGER NOT NULL
status	VARCHAR(20)	"status" VARCHAR(20) DEFAULT 'pending' CHECK("status" IN ('pending', 'approved', 'rejected'))
registered_at	DATETIME	"registered_at" DATETIME DEFAULT CURRENT_TIMESTAMP
processed_at	DATETIME	"processed_at" DATETIME
semesters		CREATE TABLE semesters (id INTEGER PRIMARY KEY AUTOINCREMENT, name VARCHAR(20) NOT NULL, year INTEGER NOT NULL, start_date DATE, end_date DATE, is_current INTEGER DEFAULT 0)
id	INTEGER	"id" INTEGER
name	VARCHAR(20)	"name" VARCHAR(20) NOT NULL
year	INTEGER	"year" INTEGER NOT NULL
start_date	DATE	"start_date" DATE
end_date	DATE	"end_date" DATE
is_current	INTEGER	"is_current" INTEGER DEFAULT 0
sqlite_sequence		CREATE TABLE sqlite_sequence(name,seq)
name		"name"
seq		"seq"
students		CREATE TABLE students (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL UNIQUE, student_code VARCHAR(20) NOT NULL UNIQUE, class_name VARCHAR(50), major VARCHAR(100), FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE)
id	INTEGER	"id" INTEGER
user_id	INTEGER	"user_id" INTEGER NOT NULL UNIQUE

Name	Type	Schema
student_code	VARCHAR(20)	"student_code" VARCHAR(20) NOT NULL UNIQUE
class_name	VARCHAR(50)	"class_name" VARCHAR(50)
major	VARCHAR(100)	"major" VARCHAR(100)
submissions		CREATE TABLE submissions (id INTEGER PRIMARY KEY AUTOINCREMENT, assignment_id INTEGER NOT NULL, student_id INTEGER NOT NULL, file_url VARCHAR(500), file_name VARCHAR(255), file_size INTEGER, progress_report TEXT, submitted_at DATETIME DEFAULT CURRENT_TIMESTAMP, is_late INTEGER DEFAULT 0, is_final INTEGER DEFAULT 0, version INTEGER DEFAULT 1, FOREIGN KEY (assignment_id) REFERENCES assignments(id) ON DELETE CASCADE, FOREIGN KEY (student_id) REFERENCES students(id))
id	INTEGER	"id" INTEGER
assignment_id	INTEGER	"assignment_id" INTEGER NOT NULL
student_id	INTEGER	"student_id" INTEGER NOT NULL
file_url	VARCHAR(500)	"file_url" VARCHAR(500)
file_name	VARCHAR(255)	"file_name" VARCHAR(255)
file_size	INTEGER	"file_size" INTEGER
progress_report	TEXT	"progress_report" TEXT
submitted_at	DATETIME	"submitted_at" DATETIME DEFAULT CURRENT_TIMESTAMP
is_late	INTEGER	"is_late" INTEGER DEFAULT 0
is_final	INTEGER	"is_final" INTEGER DEFAULT 0
version	INTEGER	"version" INTEGER DEFAULT 1
supervisions		CREATE TABLE supervisions (id INTEGER PRIMARY KEY AUTOINCREMENT, student_id INTEGER NOT NULL, lecturer_id INTEGER NOT NULL, class_id INTEGER NOT NULL, status VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active', 'completed')), start_date DATETIME DEFAULT CURRENT_TIMESTAMP, end_date DATETIME, FOREIGN KEY (student_id) REFERENCES students(id), FOREIGN KEY (lecturer_id) REFERENCES lecturers(id), FOREIGN KEY (class_id) REFERENCES classes(id))
id	INTEGER	"id" INTEGER
student_id	INTEGER	"student_id" INTEGER NOT NULL
lecturer_id	INTEGER	"lecturer_id" INTEGER NOT NULL
class_id	INTEGER	"class_id" INTEGER NOT NULL
status	VARCHAR(20)	"status" VARCHAR(20) DEFAULT 'active' CHECK("status" IN ('active', 'completed'))
start_date	DATETIME	"start_date" DATETIME DEFAULT CURRENT_TIMESTAMP
end_date	DATETIME	"end_date" DATETIME
topics		CREATE TABLE topics (id INTEGER PRIMARY KEY AUTOINCREMENT, student_id INTEGER NOT NULL, class_id INTEGER NOT NULL, title VARCHAR(500) NOT NULL, description TEXT, created_at DATETIME DEFAULT CURRENT_TIMESTAMP, updated_at DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (student_id) REFERENCES students(id), FOREIGN KEY (class_id) REFERENCES classes(id))
id	INTEGER	"id" INTEGER
student_id	INTEGER	"student_id" INTEGER NOT NULL
class_id	INTEGER	"class_id" INTEGER NOT NULL
title	VARCHAR(500)	"title" VARCHAR(500) NOT NULL
description	TEXT	"description" TEXT
created_at	DATETIME	"created_at" DATETIME DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	"updated_at" DATETIME DEFAULT CURRENT_TIMESTAMP
users		CREATE TABLE users (id INTEGER PRIMARY KEY AUTOINCREMENT, email VARCHAR(100) NOT NULL UNIQUE, password VARCHAR(255) NOT NULL, full_name VARCHAR(100) NOT NULL, role VARCHAR(20) NOT NULL CHECK (role IN ('student', 'lecturer')), phone VARCHAR(20), created_at

Name	Type	Schema
		DATETIME DEFAULT CURRENT_TIMESTAMP)
id	INTEGER	"id" INTEGER
email	VARCHAR(100)	"email" VARCHAR(100) NOT NULL UNIQUE
password	VARCHAR(255)	"password" VARCHAR(255) NOT NULL
full_name	VARCHAR(100)	"full_name" VARCHAR(100) NOT NULL
role	VARCHAR(20)	"role" VARCHAR(20) NOT NULL CHECK("role" IN ('student', 'lecturer'))
phone	VARCHAR(20)	"phone" VARCHAR(20)
created_at	DATETIME	"created_at" DATETIME DEFAULT CURRENT_TIMESTAMP

Indices (0)

Name	Type	Schema

Views (5)

Name	Type	Schema
view_available_lecturers		CREATE VIEW view_available_lecturers AS SELECT l.id AS lecturer_id, u.full_name AS lecturer_name, l.department, l.degree, lq.course_type_id, ct.type_name AS course_type_name, lq.max_students, lq.current_students, (lq.max_students - lq.current_students) AS available_slots FROM lecturer_quotas lq JOIN lecturers l ON lq.lecturer_id = l.id JOIN users u ON l.user_id = u.id JOIN course_types ct ON lq.course_type_id = ct.id WHERE lq.current_students < lq.max_students AND l.is_active = 1
lecturer_id	INTEGER	"lecturer_id" INTEGER
lecturer_name	VARCHAR(100)	"lecturer_name" VARCHAR(100)
department	VARCHAR(100)	"department" VARCHAR(100)
degree	VARCHAR(50)	"degree" VARCHAR(50)
course_type_id	INTEGER	"course_type_id" INTEGER
course_type_name	VARCHAR(50)	"course_type_name" VARCHAR(50)
max_students	INTEGER	"max_students" INTEGER
current_students	INTEGER	"current_students" INTEGER
available_slots		"available_slots"
view_can_register		CREATE VIEW view_can_register AS SELECT r.student_id, r.lecturer_id, r.course_type_id, r.semester_id, CASE WHEN EXISTS (SELECT 1 FROM registrations r2 WHERE r2.student_id = r.student_id AND r2.course_type_id = r.course_type_id AND r2.semester_id = r.semester_id AND r2.status IN ('pending', 'approved')) THEN 0 WHEN (SELECT (max_students - current_students) FROM lecturer_quotas WHERE lecturer_id = r.lecturer_id AND semester_id = r.semester_id AND course_type_id = r.course_type_id) <= 0 THEN 0 ELSE 1 END AS can_register FROM registrations r
student_id	INTEGER	"student_id" INTEGER
lecturer_id	INTEGER	"lecturer_id" INTEGER
course_type_id	INTEGER	"course_type_id" INTEGER
semester_id	INTEGER	"semester_id" INTEGER
can_register		"can_register"
view_published_reports		CREATE VIEW view_published_reports AS SELECT pr.id AS report_id, t.title, t.description AS abstract, pr.file_url, pr.source_code_url, pr.published_at,

Name	Type	Schema
		s.student_code, u.full_name AS student_name, u2.full_name AS lecturer_name, ct.type_name AS course_type, sem.name AS semester, sem.year FROM published_reports pr JOIN topics t ON pr.topic_id = t.id JOIN students s ON pr.student_id = s.id JOIN users u ON s.user_id = u.id JOIN classes c ON pr.class_id = c.id JOIN lecturers l ON c.lecturer_id = l.id JOIN users u2 ON l.user_id = u2.id JOIN courses cr ON c.course_id = cr.id JOIN course_types ct ON cr.course_type_id = ct.id JOIN semesters sem ON c.semester_id = sem.id ORDER BY pr.published_at DESC
report_id	INTEGER	"report_id" INTEGER
title	VARCHAR(500)	"title" VARCHAR(500)
abstract	TEXT	"abstract" TEXT
file_url	VARCHAR(500)	"file_url" VARCHAR(500)
source_code_url	VARCHAR(500)	"source_code_url" VARCHAR(500)
published_at	DATETIME	"published_at" DATETIME
student_code	VARCHAR(20)	"student_code" VARCHAR(20)
student_name	VARCHAR(100)	"student_name" VARCHAR(100)
lecturer_name	VARCHAR(100)	"lecturer_name" VARCHAR(100)
course_type	VARCHAR(50)	"course_type" VARCHAR(50)
semester	VARCHAR(20)	"semester" VARCHAR(20)
year	INTEGER	"year" INTEGER
view_student_grades		CREATE VIEW view_student_grades AS SELECT s.id AS student_id, s.student_code, u.full_name AS student_name, c.id AS class_id, c.class_name, ct.type_name AS course_type, cs.grade, CASE WHEN cs.grade >= 8.5 THEN 'A' WHEN cs.grade >= 7.0 THEN 'B' WHEN cs.grade >= 5.5 THEN 'C' WHEN cs.grade >= 4.0 THEN 'D' ELSE 'F' END AS grade_letter, sem.name AS semester, sem.year FROM students s JOIN class_students cs ON s.id = cs.student_id JOIN classes c ON cs.class_id = c.id JOIN courses cr ON c.course_id = cr.id JOIN course_types ct ON cr.course_type_id = ct.id JOIN semesters sem ON c.semester_id = sem.id JOIN users u ON s.user_id = u.id WHERE cs.grade IS NOT NULL
student_id	INTEGER	"student_id" INTEGER
student_code	VARCHAR(20)	"student_code" VARCHAR(20)
student_name	VARCHAR(100)	"student_name" VARCHAR(100)
class_id	INTEGER	"class_id" INTEGER
class_name	VARCHAR(100)	"class_name" VARCHAR(100)
course_type	VARCHAR(50)	"course_type" VARCHAR(50)
grade	REAL	"grade" REAL
grade_letter		"grade_letter"
semester	VARCHAR(20)	"semester" VARCHAR(20)
year	INTEGER	"year" INTEGER
view_submission_progress		CREATE VIEW view_submission_progress AS SELECT a.id AS assignment_id, a.title AS assignment_title, c.id AS class_id, c.class_name, (SELECT COUNT(*) FROM class_students WHERE class_id = c.id) AS total_students, (SELECT COUNT(*) FROM submissions WHERE assignment_id = a.id) AS submitted_count, (SELECT COUNT(*) FROM submissions WHERE assignment_id = a.id AND is_late = 1) AS late_count, ROUND((SELECT COUNT(*) FROM submissions WHERE assignment_id = a.id) * 100.0 / (SELECT COUNT(*) FROM class_students WHERE class_id = c.id), 2) AS completion_rate FROM

Name	Type	Schema
		assignments a JOIN classes c ON a.class_id = c.id
assignment_id	INTEGER	"assignment_id" INTEGER
assignment_title	VARCHAR(200)	"assignment_title" VARCHAR(200)
class_id	INTEGER	"class_id" INTEGER
class_name	VARCHAR(100)	"class_name" VARCHAR(100)
total_students		"total_students"
submitted_count		"submitted_count"
late_count		"late_count"
completion_rate		"completion_rate"

Triggers (6)

Name	Type	Schema
auto_create_supervision		CREATE TRIGGER auto_create_supervision AFTER UPDATE ON registrations FOR EACH ROW WHEN NEW.status = 'approved' AND OLD.status != 'approved' BEGIN INSERT INTO supervisions (student_id, lecturer_id, class_id, status, start_date) VALUES (NEW.student_id, NEW.lecturer_id, (SELECT id FROM classes WHERE lecturer_id = NEW.lecturer_id AND semester_id = NEW.semester_id LIMIT 1), 'active', DATETIME('now')); END
auto_increment_version		CREATE TRIGGER auto_increment_version BEFORE INSERT ON submissions FOR EACH ROW BEGIN UPDATE submissions SET version = (SELECT COALESCE(MAX(version), 0) + 1 FROM submissions WHERE assignment_id = NEW.assignment_id AND student_id = NEW.student_id) WHERE id = NEW.id; END
update_assignment_overdue		CREATE TRIGGER update_assignment_overdue AFTER UPDATE ON assignments FOR EACH ROW WHEN NEW.deadline < DATETIME('now') BEGIN UPDATE assignments SET is_overdue = 1 WHERE id = NEW.id; END
update_lecturer_counts		CREATE TRIGGER update_lecturer_counts AFTER UPDATE ON registrations FOR EACH ROW WHEN NEW.status = 'approved' AND OLD.status != 'approved' BEGIN UPDATE lecturers SET current_thesis = CASE WHEN NEW.course_type_id = 2 THEN current_thesis + 1 ELSE current_thesis END, current_project = CASE WHEN NEW.course_type_id = 1 THEN current_project + 1 ELSE current_project END WHERE id = NEW.lecturer_id; END
update_lecturer_quota_on_approve		CREATE TRIGGER update_lecturer_quota_on_approve AFTER UPDATE ON registrations FOR EACH ROW WHEN NEW.status = 'approved' AND OLD.status != 'approved' BEGIN UPDATE lecturer_quotas SET current_students = current_students + 1 WHERE lecturer_id = NEW.lecturer_id AND semester_id = NEW.semester_id AND course_type_id = NEW.course_type_id; END
update_submission_is_late		CREATE TRIGGER update_submission_is_late AFTER INSERT ON submissions FOR EACH ROW BEGIN UPDATE submissions SET is_late = CASE WHEN NEW.submitted_at > (SELECT deadline FROM assignments WHERE id = NEW.assignment_id) THEN 1 ELSE 0 END WHERE id = NEW.id; END